

How search works.

From words to results.

Build an index. Explain a ranking.

Find out why matching words can still miss the meaning.

🕒 16-20 min core reading · 20-30 min practice · 12 short sections

Before an AI assistant can answer from a collection, it needs a way to find useful material. Search turns a question into a small, ordered set of candidates. Understanding that step makes embeddings and RAG much easier to reason about.

WHAT YOU WILL UNDERSTAND

The main question: how does “space rescue” become a ranked list of movies, and why can that list still be wrong?

The practical path: analyze text, build an index, match candidates, calculate ranking scores, and judge the results against an intent.

Reading path: twelve short sections, three diagrams, a real search explorer, and a Python workbook. Expand deeper notes when you want the extra detail.

1. Start with an information need

Information retrieval (IR) finds material relevant to a need from a collection. A **query** expresses that need; a **document** is the unit we retrieve; the **corpus** is the collection. Here, one movie summary is one document. Later, it might be a paragraph from a manual.

Imagine a viewer types “**space rescue**”, meaning “a story about rescuing someone in outer space.” A search engine compares that query with the catalog and returns an ordered list. It does not have to generate new text, predict a rating, or know this viewer’s viewing history.

Relevance depends on the intent. A café with “no space for a kitchen” shares a word but does not answer this need. A film where “astronauts save a stranded pilot” may answer it without sharing either query word. These two mistakes will follow us through the chapter.

Our eight movies, summaries, and relevance judgments are **fictional teaching data**. We reuse familiar titles and catalog facts, adding new summaries for this search lesson. The index and scores are calculated, not authored outputs or calls to a search service.

2. Give documents and queries the same text rules

An **analyzer** prepares text for matching. Tokenization divides text into pieces; normalization makes selected differences equivalent. Our deliberately small analyzer lowercases and extracts runs of English letters or digits:

```
"Space, RESCUE!" → ["space", "rescue"]  
"space space rescue" → unique query terms ["space", "rescue"]
```

We keep repeated terms inside documents, because their frequency can affect ranking. We count each distinct query term once. We do not remove common words, stem words, correct spelling, or expand synonyms. Thus `rescue`, `rescues`, and `saving` are different terms. The same rules run on summaries and queries.

This is a transparent English demo, not a multilingual tokenizer or an LLM tokenizer. Titles and runtime fields are not part of its text index. See the Stanford IR book’s [tokenization explanation](#).

▼ Go deeper: stop words, stemming, synonyms, and phrases

Stop words are common terms that a system may omit. Removing them saves some work but can damage names, phrases, or negation: “not funny” should not silently become “funny.” **Stemming** reduces word forms with rules; **lemmatization** aims for a linguistic base form. Both can help matching and can also merge meanings you wanted separate.

Query expansion adds alternatives such as rescue → save. It may recover missed films but also introduce irrelevant meanings. Evaluate it on several intents. Phrase search needs order and adjacency: documents containing both “space” and “rescue” need not contain the phrase “space rescue.” A positional index can check neighboring token positions. Our workbook stores positions for inspection but implements only OR/AND term matching; quote characters are punctuation, not a phrase operator. See [positional indexes](#).

3. Build an inverted index

A simple scan checks every document for each query. An **inverted index** reverses the lookup: a term points to the IDs of documents that contain it. Each entry in this **posting list** can also store its frequency and positions.

➤ Invert the direction of lookup

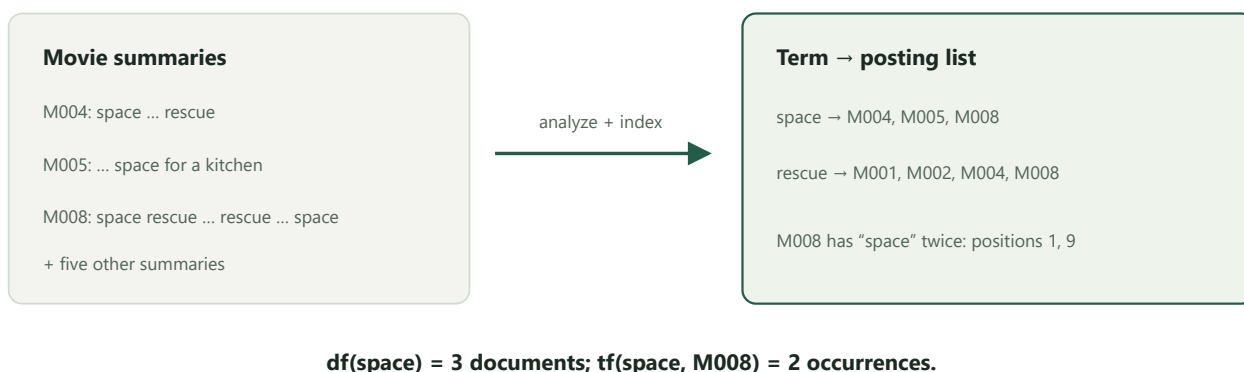


Figure 1. Start with documents containing words. Build one posting list per term so a query can jump to matching movie IDs. Counts and positions add scoring and phrase information.

For **space**, the list is M004, M005, M008. Its **document frequency**, df , is 3. Inside M008, **space** appears twice, so its **term frequency**, tf , is 2. Counting four occurrences across three documents would not make $df = 4$.

An index trades construction work and storage for faster repeated lookup. New or changed summaries need index updates; deleting a document must eventually remove its searchable content too. Our small workbook rebuilds its index on each run. A larger application typically maintains a persistent index. See [building an inverted index](#).

4. Separate matching, filters, and ranking

For **space rescue**, **OR** takes the union of the two posting lists: a movie may contain either word. **AND** takes their intersection: it must contain both. In our catalog OR finds five movies; AND finds M004 and M008. AND does not require adjacent words or an understanding of the whole request.

A **hard filter** enforces a structured requirement. “Known runtime below 120 minutes” uses the runtime field, excluding 140-minute City Sprint and unknown-runtime Unlisted Voyage. A high text score cannot override that requirement. Unknown is not the same as short.

➤ Eligibility, matching, and ordering answer different questions

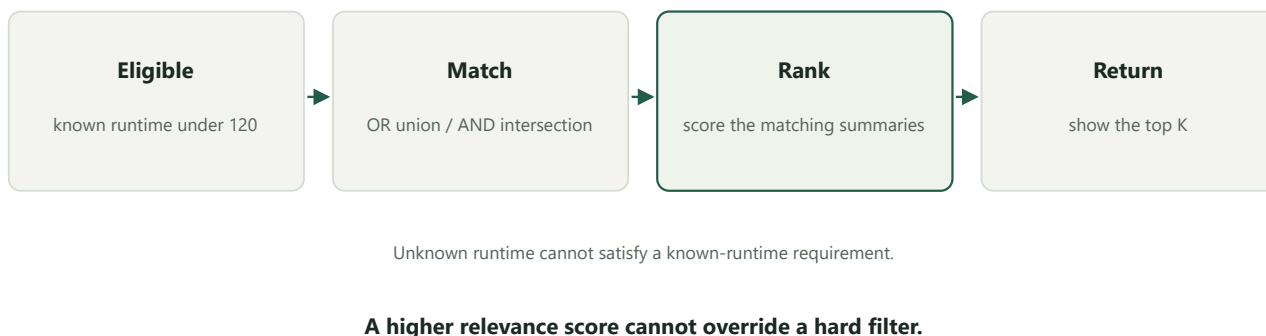


Figure 2. Runtime requirements decide eligibility. Query terms identify candidates. Ranking orders those candidates; top-K limits how many are shown. Full-catalog statistics stay fixed in our lab.

Ranking orders the eligible matches. **Top-K** returns up to K of them. Apply required filters before truncating to K; otherwise, filtering an already-short list can leave gaps while valid candidates farther down are never considered. Query words like `under 120 minutes` do not automatically become numeric filters in this demo: use its runtime control.

▼ Go deeper: fields, exact lookup, and two-stage search

A real catalog can index title, synopsis, cast, and tags separately. A title match may deserve a field boost; a movie ID may require exact lookup. Choose normalization per field instead of running every value through one analyzer. Explicit filters can narrow genre, language, availability, or access permissions without treating them as soft preferences.

Large search systems often retrieve a broad candidate set cheaply, then apply a more expensive **reranker** to that set. Candidate count and final K are different controls. A reranker can reorder what it receives; it cannot recover a relevant document excluded before that stage. Index compression, caching, and distributed partitions address scale, but they do not repair a poorly defined relevance task.

5. Understand term overlap and TF-IDF

Our first baseline gives one point per distinct query term present. M004 and M008 both match two terms, so they tie at 2. Movie ID breaks exact ties. This is easy to inspect, but it ignores frequency and whether a term is rare.

TF-IDF combines term frequency with **inverse document frequency**: repeated evidence within a document can help, while a term found almost everywhere is less discriminating. There are several variants. Our score is an unnormalized sum of log-TF weights, using natural logarithms:

$$s(q, d) = \sum_{t \in q, tf(t, d) > 0} (1 + \ln tf(t, d)) \ln \frac{N}{df(t)}.$$

N is the number of indexed documents, here 8. For M004, both term counts are 1: **space** contributes $\ln(8/3) = 0.9808$ and **rescue** contributes $\ln(8/4) = 0.6931$. Total: **1.6740**. M008 repeats both terms, so this variant scores it **2.8343**.

Absent terms contribute zero. A term in every document has zero IDF under this formula. Matching and scoring remain separate: a matching document may have a zero score. See [TF-IDF weighting](#).

▼ Go deeper: sparse vectors and score conventions

Give each vocabulary term one coordinate. A TF-IDF document vector is **sparse**: most coordinates are zero. One common alternative ranks by cosine similarity between weighted query and document vectors. That normalizes by vector lengths; our sum does not. Document and query weighting, smoothing, logarithm base, and normalization all affect numerical scores.

Do not compare a raw score from one formula with a score from another as if they shared a calibrated scale. Compare their ordered results against the same relevance judgments. Dense learned embeddings are another representation, introduced in Chapter 17; a sparse lexical vector is already a vector without being a neural embedding.

6. Improve the lexical baseline with BM25

BM25 also rewards matching terms, but adds two useful controls: repeated occurrences give **diminishing returns**, and term frequency is adjusted for **document length** relative to the collection. Repeating "space" ten times should not simply produce ten times the evidence.

Our exact teaching variant is:

$$\text{IDF}(t) = \ln \left(1 + \frac{N - df(t) + 0.5}{df(t) + 0.5} \right),$$

$$s(q, d) = \sum_{t \in q} \text{IDF}(t) \frac{tf(t, d)(k_1 + 1)}{tf(t, d) + k_1(1 - b + b|d|/\text{avgdl})}.$$

$|d|$ counts summary tokens; **avgdl** is their average across all eight documents: $94/8 = 11.75$. We use $k_1 = 1.2$ and $b = 0.75$. Larger k_1 lets extra repetitions contribute more; $b = 0$ removes length normalization. At $k_1 = 0$, a present term contributes its IDF regardless of count or length; missing terms still contribute zero.

For M004, $|d| = 10$ and both counts are 1. The frequency/length factor is $2.2/[1 + 1.2(0.25 + 0.75 \times 10/11.75)] \approx 1.0649$. Multiply each term's BM25 IDF by this factor, then sum: **1.7439**. M008 scores **2.1863**, so it ranks first.

These scores are **weights, not relevance probabilities**. We use the positive IDF form documented by [Apache Lucene](#), with explicit token counts and the formula above; we do not claim byte-for-byte score equivalence with Lucene. The [Stanford BM25 treatment](#) explains the underlying family and alternative forms.

7. Recognize what lexical scoring cannot fix

Default BM25 returns **Parallel World, Quiet Orbit, Cafe Chaos**. The first two fit the space-rescue intent. Cafe Chaos contains “space” in another sense. Unlisted Voyage fits the intent but its summary says “astronauts save a stranded pilot,” so neither query term has a posting for it.

Changing BM25 parameters can reorder matches, but cannot add a movie with no shared terms to this candidate set. Switching to AND removes Cafe Chaos, yet still misses Unlisted Voyage. Returning five results does not fix that missing candidate either.

Common failure patterns include vocabulary mismatch, ambiguous words, spelling errors, overly strict filters, stale summaries, and the needed fact being absent from the collection. Inspect the missed or incorrect movie before deciding whether to change the analyzer, metadata, query expansion, or retrieval method. A more elaborate scorer is not a universal repair.

8. Judge the ranked list against the intent

The ranking algorithm never reads our relevance labels. Separately, we authored binary judgments for each of three preset intents across all eight movies. For `space rescue`, M004, M007, and M008 are relevant; the other five are nonrelevant.

With default BM25 and $K = 3$, two returned movies are relevant out of three relevant movies in the corpus:

Measure	This chapter's definition	Default result
Precision@K	Relevant results in the first K slots / K	2/3 = 66.67%
Recall@K	Relevant results in those slots / relevant eligible movies	2/3 = 66.67%
Reciprocal rank@K	1 / rank of the first relevant result; 0 if absent in top K	1/1 = 1

We count empty result slots as zero for Precision@K. With AND, only two movies are returned: both are relevant, but **Precision@3 stays 2/3** under that convention. Precision among returned items would be 2/2; it answers a different question.

The runtime filter removes unknown-runtime M007 from eligibility. Recall becomes $2/2 = 100\%$ **for that smaller task**, not because the search discovered the missed movie. We show no relevance metrics for arbitrary queries without judgments. When there are no relevant eligible documents, recall is undefined. See [evaluating ranked results](#).

▼ Go deeper: query sets, MRR, MAP, and NDCG

MRR averages reciprocal rank over queries, so it emphasizes the first useful result. Our explorer shows one query's truncated reciprocal rank, not a dataset-wide MRR. **Average precision** rewards retrieving relevant documents early across a ranking; **MAP** averages that measure over queries. Specify cutoffs and missing-result conventions.

NDCG supports graded relevance. A common gain is $2^{\text{grade}} - 1$, discounted by $\log_2(\text{rank} + 1)$. Sum to K and divide by the ideal ranking's sum. If the ideal sum is zero, choose and document a reporting convention. These additional metrics are explained here, not implemented in the workbook.

Three authored queries cannot establish production quality. Use representative query sets and independently judged results; tune on development queries and reserve others for evaluation. Unjudged is not inherently nonrelevant, and clicks can reflect position or presentation rather than true relevance. Report per-intent failures along with averages, latency, and coverage.

9. Explore the search process

Start with the default query. Open the inverted index and each result's score explanation. Switch between overlap, TF-IDF, and BM25: the order changes, but the default three-slot relevance metrics are identical. A different list is not automatically a better list.

FOLLOW THE WORDS. EXPLAIN THE RANK.

Real search · fictional catalog

One query. Three ways to rank.

Search eight authored movie summaries locally. Titles label the results; only summary words are indexed. No model or external service is called.

Default query: space rescue. OR matching, no runtime limit, K = 3. Eight summaries have 94 tokens in total (average 11.75). Space occurs in 3 documents; rescue in 4.

Method	Top three movie IDs (scores)
Term overlap	M004 (2.0000) → M008 (2.0000) → M001 (1.0000)
Log TF-IDF	M008 (2.8343) → M004 (1.6740) → M005 (0.9808)
BM25	M008 (2.1863) → M004 (1.7439) → M005 (0.8759)

Titles: M008 = Parallel World; M004 = Quiet Orbit; M005 = Cafe Chaos; M001 = Moonlight Map. Relevant IDs are M004, M007, M008, so all three methods get Precision@3 = Recall@3 = 66.67% and reciprocal rank@3 = 1.

BM25 retrieves a cafe's "space" but misses Unlisted Voyage: "Astronauts save a stranded pilot beyond a distant moon." AND removes partial matches but still misses M007. Runtime below 120 excludes M002 (140 min) and M007 (unknown); recall becomes 100% only within that smaller eligible set.

Try AND, then the runtime filter, and inspect what changed in the metric denominator. Reset before comparing. Finally try `hilarious quest`: no words match. The explorer does not silently add synonyms or produce a made-up recommendation.

The corpus statistics stay fixed across filters, and ties use ascending movie ID. Repeated query terms count once. Results are bounded by K; a blank query returns none. The JSON download records settings, postings, contributions, and the preset intent and evaluation metrics when available.

10. Run the workbook and practice

Extract its four files together. Python 3.10 or newer is enough; no packages, credentials, or API calls are needed.

```
py search.py
py search.py --method overlap
py search.py --mode all
py search.py --limit under120 --k 5
py search.py --query "funny adventure" --method tfidf
py -m unittest -v test_search.py
```

Use `python` or `python3` if that is your installed command. The script reads `movies.json` beside itself and prints JSON without changing input files. `--input` selects another compatible catalog; invalid data or settings stop with exit code 2. `--k1` and `--b` adjust BM25. Like the browser, it performs lexical search; it does not train a model or generate text.

Exercise 1: work with posting lists

`space` maps to M004, M005, M008. `rescue` maps to M001, M002, M004, M008. Find their OR and AND candidates. Does AND require an exact phrase?

▼ Reveal the reasoning

OR: M001, M002, M004, M005, M008. AND: M004, M008. AND needs both terms somewhere in the summary; it says nothing about order or adjacency. M004 has "space crew plans a rescue," so it satisfies AND without containing the phrase "space rescue."

Exercise 2: explain the three-slot result

The default ranking is M008, M004, M005. Relevant IDs are M004, M007, M008. Calculate Precision@3, Recall@3, and reciprocal rank@3. What happens to recall when the runtime requirement excludes M007?

▼ Reveal the reasoning

Two retrieved results are relevant: precision and recall are both 2/3. The first result is relevant, so reciprocal rank is 1. Filtering out M007 changes the relevant eligible set to two movies, making recall 2/2. This is a change in the task's eligibility, not evidence that the semantic miss has been repaired.

Exercise 3: repair a missed candidate deliberately

In a copy of the catalog, add the truthful-to-this-fictional-story phrase `space rescue` to M007's summary. Run the query with no runtime filter, then with `--limit under120`. Predict what changes before running it.

▼ Reveal the reasoning

M007 now enters the lexical candidate set without a runtime filter, although a small K can still omit it after ranking. Adding terms also changes document frequencies, the average length, and therefore other BM25 scores. With the runtime requirement it remains excluded because its runtime is unknown. Restore the original file to reproduce handbook numbers. In a real system, enrich metadata only when sources support it; do not stuff unrelated query terms into summaries.

11. Check your understanding

01 What does an inverted index map?

- A. Movie IDs to predicted star ratings.
- B. Terms to the documents containing them.
- C. Queries to answers written by an LLM.

Answer: B. A posting list lets the engine look up matching document IDs by term, often with frequencies and positions.

02 Space occurs twice in M008 and appears in three documents. What are tf and df?

- A. $tf = 2$ for M008; $df = 3$ for the collection.
- B. $tf = 3$; $df = 2$.
- C. Both are 5.

Answer: A. Term frequency counts occurrences within one document; document frequency counts distinct documents containing the term.

03 Which statement about a BM25 score of 2.18 is correct?

- A. It means a 218% probability of relevance.
- B. It guarantees the viewer will enjoy the movie.
- C. It is a lexical ranking weight under a particular query, index, and formula.

Answer: C. BM25 scores help order matches. They are not calibrated probabilities or proof of usefulness.

04 Two relevant movies fill two of three requested result slots. There are three relevant eligible movies. Under our convention, what is Precision@3?

- A. 100%, because both returned movies are relevant.
- B. 2/3, because the denominator is the three requested slots.
- C. 3/2, because three movies are relevant.

Answer: B. Precision@K uses K here, with empty slots counted as zero. Precision among returned results would answer a different question.

05 A summary says “astronauts save a stranded pilot” but the query is “space rescue”. Why can BM25 miss it?

- A. The runtime must be too long.
- B. More copies of the query always fix this.
- C. There may be no shared analyzed terms; weighting cannot create missing candidates.

Answer: C. Vocabulary mismatch needs a change such as query expansion or semantic retrieval, with evaluation. Repeating words cannot create a lexical match.

12. Connect search to embeddings and RAG

You can now trace **need** → **analyzed query** → **posting lists** → **eligible candidates** → **ranking** → **top-K** → **relevance evaluation**. Keep each boundary visible when something goes wrong.

➤ Retrieval supplies evidence; generation writes an answer

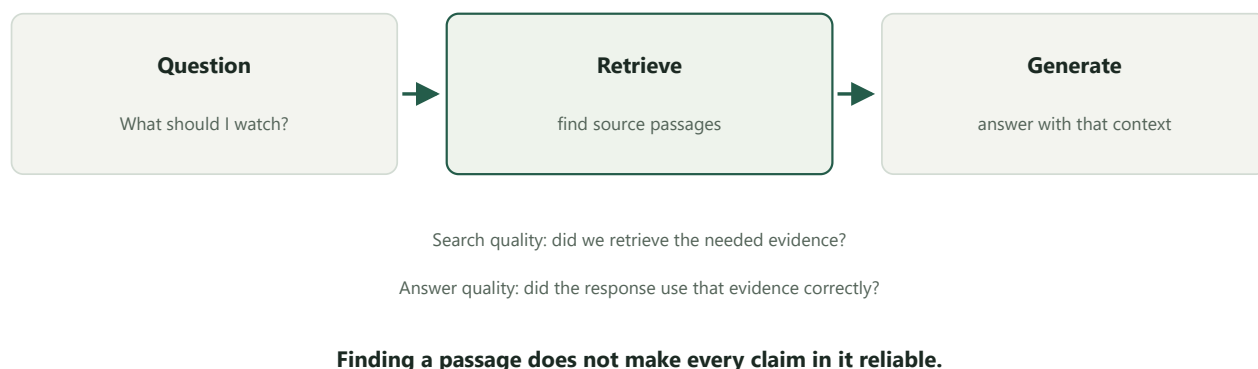


Figure 3. In a future RAG application, retrieve useful source passages first, then provide them with the question to a generator. Check that the answer is supported by those sources.

In **retrieval-augmented generation (RAG)**, a generator receives retrieved material as context for answering. Retrieval can use keywords, embeddings, or a combination; RAG does not inherently require a vector database. A useful retrieved passage can support an answer, but retrieval alone does not guarantee source quality, factual correctness, or faithful use of the passage.

Chapter 17: Embeddings and vector search tackles matching beyond literal shared words. We will keep this keyword baseline so that a new representation has something concrete to improve upon.

▼ Quick-reference glossary

Term	Meaning here
Query / corpus / document	Search input / collection / retrievable unit
Analyzer	Tokenization and normalization rules
Inverted index / posting	Term-to-document lookup / one document's entry
tf / df	Occurrences in one document / documents containing a term
OR / AND	Union / intersection of term matches
Filter / rank / top-K	Enforce eligibility / order matches / limit returned results
TF-IDF	Weight local frequency by collection rarity
BM25	Lexical weighting with saturation and length normalization
Relevance judgment	Assessment against a stated information need
Reranker	A second scorer applied to retrieved candidates
RAG	Generate an answer using retrieved context

Sources and further reading

The primary references are linked beside the concepts above. For a complete treatment, the authors' [Introduction to Information Retrieval](#) covers indexing, ranked retrieval, and evaluation. The formulas, analyzer choices, fictional catalog, examples, and score conventions used in this chapter are explicitly documented so you can reproduce them.